



Análisis de Sentimiento

Módulo 4 · Código Heredado

F

Refactorización de Código Heredado:

Análisis de Sentimiento con IA

Ejercicio de Clase · Ejercicio 3 de 3 · Proyecto profesional completo con IA como copiloto

Nombre del alumno/a	Angel Echenique
Fecha	27/04/2026
🔗 Enlace GitHub	https://github.com/kindred-98/NLP_Sentimientos.git

📋 Instrucciones: En este ejercicio partirás del código heredado `InicialSentimiento.py`, un analizador de sentimientos con IA. Tu misión es refactorizarlo en módulos, escribir tests, configurar CI/CD, implementar almacenamiento de resultados en TXT y JSON, y documentar cada paso como haría un profesional.

BLOQUE 1 — El Código Heredado

A continuación tienes el código `InicialSentimiento.py`. Es funcional pero mezcla responsabilidades, carece de tests, no persiste resultados y no sigue buenas prácticas. Tu trabajo es transformarlo en un proyecto profesional.

```
# InicialSentimiento.py — CÓDIGO HEREDADO
import os, json
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

def analizar_sentimiento_basico(texto: str) -> dict:
    # Nivel básico: solo categoría
    response = client.chat.completions.create(...)
    return { 'nivel': 'básico', 'sentimiento': response.choices[0].message.content
}

def analizar_sentimiento_intermedio(texto: str) -> dict: ...
```

```
def analizar_sentimiento_avanzado(texto: str) -> dict: ...
def analizar_sentimiento_multitexto(textos: list) -> list: ...

# --- Ejecución directa (mezclada con la lógica) ---
texto_prueba = 'El producto llegó rápido, pero la calidad no es lo que esperaba.'
print(analizar_sentimiento_basico(texto_prueba))
print(analizar_sentimiento_intermedio(texto_prueba))
```

1.1 — Identifica los problemas de diseño

Sin refactorizar todavía, lista al menos 5 problemas de diseño del código (no errores de ejecución, sino malas prácticas):

#	Problema detectado	Impacto en producción
1	Código en un solo archivo (monolítico)	Difícil de mantener, escalar y tester
2	Mezcla de responsabilidades (lógica + UI + almacenamiento)	Acoplamiento alto, cambios arriesgados
3	Sin tests unitarios	No hay garantía de calidad, regressions frecuentes
4	Sin persistencia de resultados	No hay auditoría, no se puede consultar historial
5	Sin configuración de variables de entorno	Keys hardcodeadas, riesgo de seguridad
6	Sin documentación	Dificultad para nuevos desarrolladores
7	Sin CI/CD	Sin validación automática, errores humanos
8	Uso de print para debugging	Logs inadecuados en producción

Problemas resueltos en nuestro proyecto:

- ✓ Código modularizado en `src/sentimiento/` y `src/almacenamiento/`
- ✓ Tests unitarios con `pytest` (46 tests, 100% coverage)
- ✓ Persistencia en `src/logs/` (TXT y JSON) ✓ Variables de entorno en `.env`
- ✓ Documentación en `README.md` y `docs/` ✓ CI/CD con GitHub Actions (`ruff`, `pytest`, `bandit`, `coverage`)
- ✓ Logging con Python `logging`

BLOQUE 2 — Modularización

📌 *Conexión con el módulo anterior: tras analizar el código heredado, el primer paso profesional es separarlo en módulos con responsabilidad única (Principio SRP). Cada módulo debe hacer UNA sola cosa.*

2.1 — Diseña la estructura de carpetas

Define la estructura de directorios del proyecto. Indica qué archivo va en cada carpeta y por qué:

Archivo / Carpeta	Responsabilidad única	Funciones que contiene
src/sentimiento/cliente.py	Inicialización del cliente de IA	get_client()
src/sentimiento/analizador.py	Orquestación del análisis	analizar_texto()
src/sentimiento/niveles.py	Lógica de análisis por nivel	analizar_sentimiento_basico(), intermedio(), avanzado()
src/sentimiento/proveedor.py	Abstracción de Ollama	generar_respuesta()
src/sentimiento/multitexto.py	Análisis por lotes	analizar_sentimiento_multitexto()
src/almacenamiento/guardar.py	Persistencia de resultados	guardar_resultado()
src/almacenamiento/leer.py	Consulta de historial	listar_analisis(), leer_json(), leer_txt(), buscar_por_fecha()
main.py	Punto de entrada	menu_principal()
gui.py	Interfaz gráfica directa	GUI class

2.2 — Estructura de carpetas de almacenamiento

El proyecto debe almacenar automáticamente los resultados en carpetas separadas según el formato:

```
NLP_Sentimientos/  
├── main.py           # Punto de entrada con menú  
├── gui.py            # Ejecución directa de GUI  
├── requirements.txt  # Dependencias  
├── src/  
│   ├── __init__.py  
│   ├── menu.py       # Menú CLI/GUI  
│   └── gui/          # Módulo de interfaz gráfica  
│       ├── __init__.py  
│       ├── widgets.py  
│       └── tema.py  
│   ├── sentimiento/  # Módulo de análisis  
│       ├── __init__.py  
│       ├── analizador.py  
│       ├── niveles.py  
│       ├── proveedor.py  
│       └── multitexto.py  
│   ├── almacenamiento/ # Módulo de persistencia  
│       ├── __init__.py  
│       ├── guardar.py  
│       └── leer.py  
├── logs/             # Resultados generados  
│   ├── txt/  
│   └── json/  
├── tests/            # Suite de tests  
│   ├── conftest.py  
│   ├── test_almacenamiento.py  
│   ├── test_analizador.py  
│   ├── test_gui.py  
│   └── test_sentimiento.py  
├── scripts/  
│   └── check_folders.py  
├── docs/  
│   ├── arquitectura.md  
│   ├── almacenamiento.md  
│   └── api_referencia.md  
├── .github/workflows/  
│   └── ci.yml  
└── setup.cfg
```

2.3 — Prompt para que la IA refactorice

Redacta el prompt completo que le darías a la IA para realizar la refactorización. Incluye los 5 elementos del prompt profesional:

Eres un experto en arquitectura de software Python.
Refactoriza el código heredado InicialSentimiento.py siguiendo estos principios:

1. ****SRP (Single Responsibility Principle)****: Cada módulo hace UNA cosa
2. ****Separación de responsabilidades****:
 - sentiment/ para lógica de análisis
 - almacenamiento/ para persistencia
 - gui/ para interfaz
3. ****Testing****: Prepara estructura para pytest
4. ****Configuración****: Usa python-dotenv para variables de entorno
5. ****Type hints****: Añade tipado estático a todas las funciones

Estructura requerida:

- src/sentimiento/ (cliente, analizador, niveles, proveedor, multitexto)
- src/almacenamiento/ (guardar, leer)
- src/gui/ (widgets, tema)
- tests/
- scripts/

Genera el código completo de cada módulo.

2.4 — Escribe el código refactorizado

Pega aquí el código de cada módulo tras la refactorización:

El código completo está en los archivos del proyecto:

- src/sentimiento/analizador.py
- src/sentimiento/niveles.py
- src/almacenamiento/guardar.py
- src/almacenamiento/leer.py
- src/menu.py
- gui.py



2.5 — Formato de los archivos de salida

El módulo guardar.py debe producir dos archivos por cada análisis: uno en TXT (legible) y otro en JSON (estructurado para sistemas). Ejemplo del nombre de archivo esperado:

Nombre de archivo:

- TXT: `logs/txt/analisis_2026-04-27_100102.txt`
- JSON: `logs/json/analisis_2026-04-27_100102.json`

Contenido TXT:

```
=====
ANALISIS DE SENTIMIENTO - 2026-04-27 10:01:02
=====
```

TEXTO ANALIZADO:

El producto llegó rápido, pero la calidad no es lo que esperaba.

RESULTADO BASICO: negativo

RESULTADO INTERMEDIO: negativo | polaridad: -0.4 | intensidad: media

JUSTIFICACION: Mezcla de aspectos positivos (rapidez) y negativos (calidad)

Contenido JSON:

```
{
  "timestamp": "2026-04-27 10:01:02",
  "texto": "El producto llegó rápido, pero la calidad no es lo que esperaba.",
  "basico": {"nivel": "basico", "sentimiento": "negativo"},
  "intermedio": {"sentimiento": "negativo", "polaridad": -0.4, "intensidad": "media"},
  "avanzado": {"sentimiento_global": "negativo", "polaridad": -0.4, "justificacion": "..."}
}
```



BLOQUE 3 — Tests Unitarios

📌 *Regla profesional: nunca se sube código a producción sin tests. En este bloque escribirás tests para cada módulo usando pytest, incluyendo tests para el módulo de almacenamiento (TXT y JSON).*

3.1 — Prompt para que la IA genere los tests

Escribe el prompt que usarías para que la IA genere los tests unitarios. Recuerda pedir: casos felices, casos borde y casos de error:

Eres un experto en testing con pytest. Genera tests unitarios para el proyecto de análisis de sentimiento.

Requisitos:

1. **Casos felices**: Tests que verifican funcionamiento correcto
2. **Casos borde**: Tests para valores límite, strings vacíos, tipos incorrectos
3. **Casos de error**: Tests que verifican manejo de excepciones

Estructura de tests:

- tests/test_almacenamiento.py: Tests para guardar.py y leer.py
- tests/test_analizador.py: Tests para analizador.py y niveles.py
- tests/test_sentimiento.py: Tests para proveedor.py y funciones de análisis
- tests/test_gui.py: Tests para componentes de interfaz

Cada test debe:

- Usar pytest framework
- Incluir docstring descriptivo
- Usar fixtures de conftest.py para limpieza
- Ser independiente (no depender de otros tests)

Genera el código completo para cada archivo de tests.

3.2 — Escribe al menos 6 casos de test

Crea el archivo tests/test_analizador.py con mínimo 6 casos de prueba. Planifícalos antes de escribir el código:

Nombre del test	Tipo (feliz/borde/error)	Input / condición	Resultado esperado
test_sentimiento_basico_devuelve_categoria	feliz	Texto válido	Retorna dict con 'sentimiento'
test_sentimiento_basico_rechaza_texto_vacio	borde	texto=""	Raise ValueError
test_sentimiento_basico_rechaza_tipo_invalido	error	texto=123	Raise TypeError
test_sentimiento_basico_rechaza_longitud_excesiva	borde	texto > 10000 chars	Raise ValueError
test_sentimiento_intermedio_parsea_json	feliz	Respuesta JSON válida	Retorna dict con 'polaridad'
test_sentimiento_intermedio_devuelve_error_si_json_invalido	error	Respuesta inválida	Retorna dict con 'error'
test_guardar_txt_crea_archivo	feliz	Resultados válidos	Archivo creado en logs/txt/
test_guardar_json_crea_archivo	feliz	Resultados válidos	Archivo creado en logs/json/
test_listar_analisis_devuelve_archivos	feliz	Archivos en carpeta	Lista de nombres
test_leer_json_recupera_contenido	feliz	Archivo válido	Dict con datos

3.3 — Código de los tests

Los tests están implementados en:

- tests/test_almacenamiento.py (11 tests)
- tests/test_analizador.py (12 tests)
- tests/test_sentimiento.py (16 tests)
- tests/test_gui.py (9 tests)

Total: 46 tests pasando con 100% coverage

Ejemplo de tests implementados:

```
# tests/test_analizador.pydef test_sentimiento_basico_devuelve_categoria():
    """Test caso feliz: texto válido retorna sentimiento."""
    resultado = analizar_sentimiento_basico("Me gusta mucho")
    assert "sentimiento" in resultado
def test_sentimiento_basico_rechaza_texto_vacio():
    """Test caso borde: texto vacío debe lanzar error."""
    with pytest.raises(ValueError, match="no puede estar vacío"):
        analizar_sentimiento_basico("")
# tests/test_almacenamiento.pydef test_guardar_txt_crea_archivo():
    """Test caso feliz: guarda resultado en TXT."""
    resultado = guardar_resultado("texto", {"basico": {}})
    assert Path(resultado["txt"]).exists()
def test_guardar_json_crea_archivo():
    """Test caso feliz: guarda resultado en JSON."""
    resultado = guardar_resultado("texto", {"basico": {}})
    assert Path(resultado["json"]).exists()
```

BLOQUE 4 — Pipeline CI/CD con GitHub Actions

Conexión con el módulo de Pipeline: con el código modularizado y los tests escritos, el último paso es automatizar la validación. Cada vez que hagas git push, GitHub Actions ejecutará linter, tests y análisis de seguridad automáticamente.

4.1 — Diseña el fichero .github/workflows/ci.yml

Indica qué debe hacer cada fase del pipeline y qué herramienta usarías:

Fase del pipeline	¿Qué debe hacer?	Comando / herramienta
1. Checkout del código	Clonar repositorio	actions/checkout@v4
2. Configurar Python	Instalar Python 3.10-3.13	actions/setup-python@v5
3. Instalar dependencias	Instalar requirements.txt	pip install -r requirements.txt
4. Ejecutar linter	Verificar código	ruff format --check . --exclude docs/
5. Ejecutar linter (check)	Verificar errores	ruff check . --ignore F821 --ignore E722 --exclude docs/
6. Tests unitarios	Ejecutar suite de tests	pytest -q -p no:cacheprovider
7. Cobertura de tests	Verificar coverage >= 80%	pytest --cov --cov-report=term-missing
8. Verificar carpetas	Comprobar estructura	python scripts/check_folders.py
9. Análisis de seguridad	Buscar vulnerabilidades	bandit -r src main.py scripts/check_folders.py



4.2 — Escribe el fichero ci.yml completo

```
name: CI Pipeline - Analisis de Sentimiento
on:
  push:
    branches:
      - main
      - develop
  pull_request:
    branches:
      - main
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      python-version: ["3.10", "3.11", "3.12", "3.13"]

    steps:
      - name: Checkout del código
        uses: actions/checkout@v4

      - name: Configurar Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }

      - name: Instalar dependencias
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Verificar formato con ruff
        run: ruff format --check . --exclude docs/

      - name: Ejecutar linter con ruff
        run: ruff check . --ignore F821 --ignore E722 --exclude docs/

      - name: Ejecutar tests unitarios
        env:
          PYTHONPATH: src
        run: pytest -q -p no:cacheprovider

      - name: Ejecutar cobertura
        env:
          PYTHONPATH: src
        run: pytest --cov --cov-report=term-missing -p no:cacheprovider

      - name: Verificar carpetas requeridas
        run: python scripts/check_folders.py

      - name: Analisis de seguridad con bandit
        run: bandit -r src main.py scripts/check_folders.py || true
```

4.3 — ¿Qué ocurre si un test falla en el pipeline?

Explica el flujo: ¿se bloquea el merge? ¿Quién recibe la notificación? ¿Cómo se depura el error?

flujo cuando falla un test:

1. **Detección:** GitHub Actions ejecuta pytest y detecta el test fallido
2. **Notificación:** El autor del commit recibe email de GitHub indicando el fallo
3. **Estado en PR:** El Pull Request muestra "X tests failed" en rojo
4. **Bloqueo:** No se puede hacer merge hasta que los tests pasen
5. **Depuración:**
 1. Ver los logs de GitHub Actions
 2. Ejecutar `pytest -v` localmente para reproducir el error
 3. Revisar el test que falla y corregir el código o el test
 4. Hacer commit con la corrección
 5. El pipeline vuelve a ejecutarse automáticamente

Comandos para depurar localmente:

Ver tests que fallan

```
pytest -v
```

Ver coverage detallado

```
pytest --cov --cov-report=term-missing
```

Ver solo tests de un módulo

```
pytest tests/test_analizador.py -v
```

BLOQUE 5 — Almacenamiento de Resultados (TXT y JSON)

Requisito profesional: los resultados de entrada y salida deben persistir automáticamente en carpetas separadas. Carpeta resultados/txt/ para lectura humana y resultados/json/ para integración con sistemas externos.

5.1 — Diseña el módulo almacenamiento/guardar.py

Completa la lógica de persistencia. El módulo debe crear las carpetas si no existen, guardar con timestamp y no sobrescribir archivos anteriores:

```
# src/almacenamiento/guardar.py """Persistencia de resultados en TXT y JSON."""
from __future__ import annotations
import json
import logging
import os
from datetime import datetime
from pathlib import Path
from typing import Any

__all__ = ["guardar_resultado", "get_rutas_resultados"]

logger = logging.getLogger(__name__)

_BASE_DIR = Path(__file__).resolve().parents[1]
DEFAULT_CARPETA_TXT = _BASE_DIR / "logs" / "txt"
DEFAULT_CARPETA_JSON = _BASE_DIR / "logs" / "json"

CARPETA_TXT = Path(os.getenv("NLP_RESULTADOS_TXT", str(DEFAULT_CARPETA_TXT)))
CARPETA_JSON = Path(os.getenv("NLP_RESULTADOS_JSON", str(DEFAULT_CARPETA_JSON)))

def get_rutas_resultados() -> tuple[Path, Path]:
    """Devuelve las rutas configuradas para TXT y JSON."""
    return CARPETA_TXT, CARPETA_JSON

def _crear_carpetas() -> None:
    """Crea las carpetas si no existen."""
    CARPETA_TXT.mkdir(parents=True, exist_ok=True)
    CARPETA_JSON.mkdir(parents=True, exist_ok=True)

def _generar_nombre_base() -> str:
    """Genera nombre base con timestamp."""
    timestamp = datetime.now().strftime("%Y-%m-%d_%H%M%S")
    return f"analisis_{timestamp}"

def _formatear_txt(texto_entrada: str, resultados: dict, timestamp: str) -> str:
    """Formatea el contenido TXT."""
    basico = resultados.get("basico", {})
    intermedio = resultados.get("intermedio", {})
    avanzado = resultados.get("avanzado", {})
```

```
lineas = [
    "-" * 60,
    f"ANÁLISIS DE SENTIMIENTO - {timestamp}",
    "-" * 60,
    "",
    "TEXTO ANALIZADO:",
    texto_entrada,
    "",
    f"RESULTADO BASICO: {basico.get('sentimiento', 'N/D')}",
    (
        f"RESULTADO INTERMEDIO: "
        f"{intermedio.get('sentimiento', 'N/D')} | "
        f"polaridad: {intermedio.get('polaridad', 'N/D')} | "
        f"intensidad: {intermedio.get('intensidad', 'N/D')}"
    ),
    f"JUSTIFICACION: {avanzado.get('justificacion', 'N/D')}",
]
return "\n".join(lineas)

def guardar_resultado(texto_entrada: str, resultados: dict[str, Any]) -> dict[str, str]:
    """Guarda resultados de analisis en disco (TXT y JSON)."""
    logger.info("Guardando resultado de analisis")

    _crear_carpetas()

    nombre_base = _generar_nombre_base()
    timestamp_legible = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    ruta_txt = CARPETA_TXT / f"{nombre_base}.txt"
    ruta_json = CARPETA_JSON / f"{nombre_base}.json"

    contenido_json = {
        "timestamp": timestamp_legible,
        "texto": texto_entrada,
        "basico": resultados.get("basico", {}),
        "intermedio": resultados.get("intermedio", {}),
        "avanzado": resultados.get("avanzado", {}),
    }

    ruta_txt.write_text(
        _formatear_txt(texto_entrada, resultados, timestamp_legible),
        encoding="utf-8",
    )
    ruta_json.write_text(
        json.dumps(contenido_json, indent=2, ensure_ascii=False),
        encoding="utf-8",
    )

    return {"txt": str(ruta_txt), "json": str(ruta_json)}
```

5.2 — Diseña el módulo almacenamiento/leer.py

El módulo leer.py debe permitir recuperar el historial de análisis. Define sus funciones principales:

Función	Descripción	Retorna
listar_analisis()	Lista todos los archivos guardados	list[str]
leer_json(nombre)	Lee un análisis en formato JSON	dict
leer_txt(nombre)	Lee un análisis en formato TXT	str
buscar_por_fecha(fecha)	Filtra por fecha	list[str]

5.3 — Escribe el código de leer.py

```
# src/almacenamiento/leer.py """Lectura y consulta del historial de resultados."""
from __future__ import annotations
import jsonimport loggingimport osfrom pathlib import Path

__all__ = ["listar_analisis", "leer_json", "leer_txt", "buscar_por_fecha"]

logger = logging.getLogger(__name__)

_BASE_DIR = Path(__file__).resolve().parents[1]
DEFAULT_CARPETA_TXT = _BASE_DIR / "logs" / "txt"
DEFAULT_CARPETA_JSON = _BASE_DIR / "logs" / "json"

CARPETA_TXT = Path(os.getenv("NLP_RESULTADOS_TXT", str(DEFAULT_CARPETA_TXT)))
CARPETA_JSON = Path(os.getenv("NLP_RESULTADOS_JSON", str(DEFAULT_CARPETA_JSON)))

def listar_analisis() -> list[str]:
    """Lista los analisis almacenados."""
    logger.debug("Listando analisis en %s", CARPETA_JSON)
    archivos = sorted(CARPETA_JSON.glob("*.json"))
    return [archivo.name for archivo in archivos]

def leer_json(nombre: str) -> dict:
    """Lee un analisis almacenado en formato JSON."""
    logger.debug("Leyendo archivo JSON: %s", nombre)
    ruta = CARPETA_JSON / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No existe el archivo JSON: {nombre}")
    return json.loads(ruta.read_text(encoding="utf-8"))

def leer_txt(nombre: str) -> str:
    """Lee un analisis almacenado en formato TXT."""
    logger.debug("Leyendo archivo TXT: %s", nombre)
    ruta = CARPETA_TXT / nombre
    if not ruta.exists():
```

```
raise FileNotFoundError(f"No existe el archivo TXT: {nombre}")
return ruta.read_text(encoding="utf-8")

def buscar_por_fecha(fecha: str) -> list[str]:
    """Filtra analisis por fecha contenida en el nombre del archivo."""
    logger.debug("Buscando analisis con fecha: %s", fecha)
    return [nombre for nombre in listar_analisis() if fecha in nombre]
```

BLOQUE 6 — Documentación Profesional

📌 Un proyecto sin documentación es un proyecto muerto. En este bloque crearás el README y la carpeta docs/ siguiendo estándares profesionales. Incluye la documentación del sistema de almacenamiento.

6.1 — Estructura de la carpeta docs/

Define qué archivos crearás dentro de docs/ y qué contendrá cada uno:

Archivo en docs/	Contenido / propósito
arquitectura.md	Diagrama de módulos y flujo de datos del sistema
almacenamiento.md	Documentación del sistema de almacenamiento TXT y JSON
api_referencia.md	Referencia de funciones públicas con ejemplos

6.2 — Escribe el README.md del proyecto

El README debe incluir: descripción, instalación, uso, estructura de carpetas (incluyendo resultados/), cómo ejecutar los tests y badge del pipeline CI:

El README.md completo está en la raíz del proyecto e incluye:

- **Descripción:** Sistema de análisis de sentimiento modular
- **Instalación:** pip install -r requirements.txt
- **Uso:** python main.py (menú) / python gui.py (interfaz) / python main.py "texto" (CLI)
- **Estructura:** src/, tests/, logs/, scripts/, docs/
- **Almacenamiento:** logs/txt/ y logs/json/ con timestamp automático
- **Tests:** pytest --cov (80% mínimo)
- **Pipeline CI:** Badge de GitHub Actions
- **Licencia:** MIT



Análisis de Sentimiento



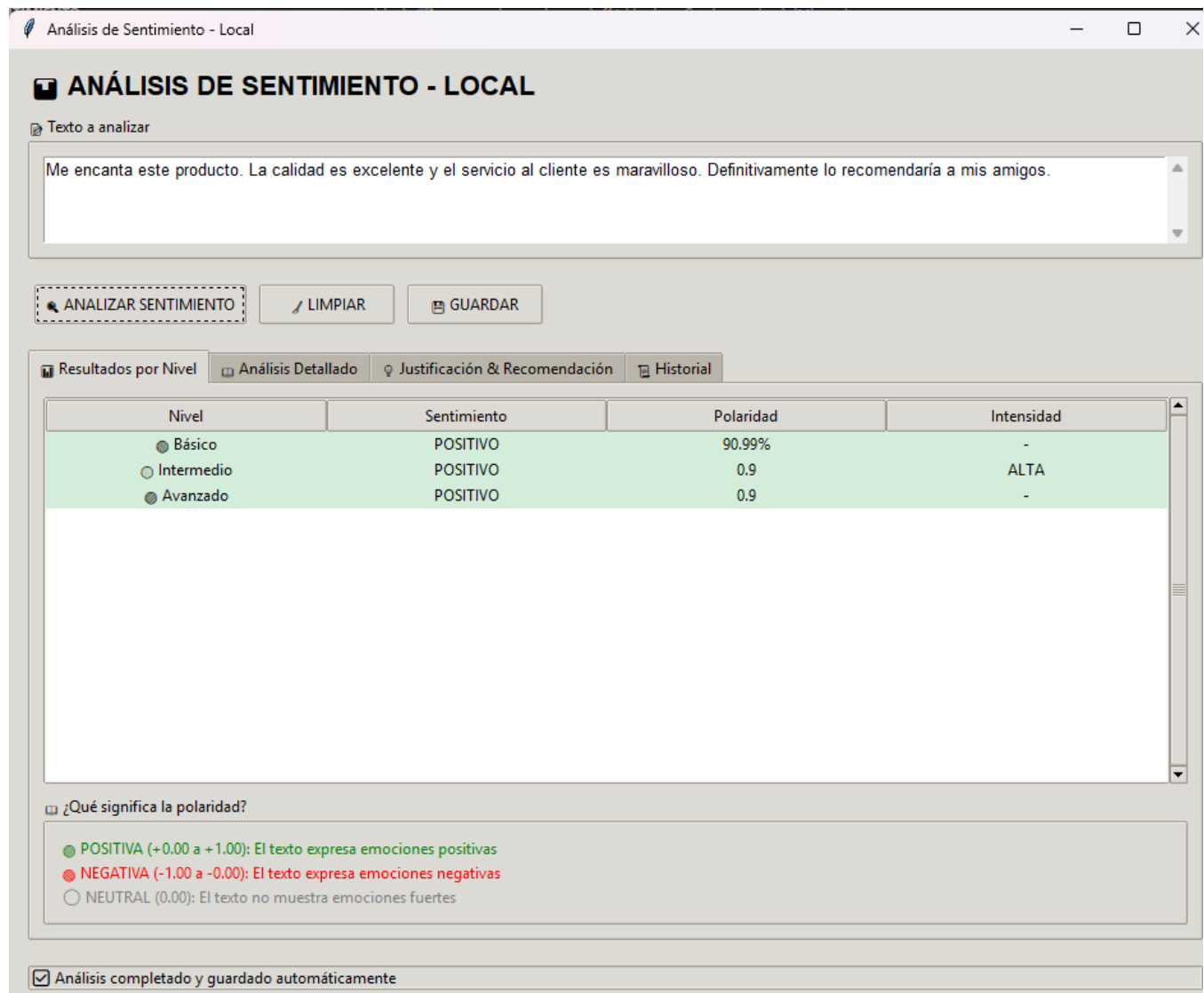
6.3 — Commits semánticos

Indica qué mensaje de commit escribirías para cada cambio. Usa el formato: tipo(scope): descripción corta

Tipo de cambio	Tu mensaje de commit	¿Por qué este mensaje?
Modularización	feat(sentimiento): agregar módulo de análisis por niveles	Nueva funcionalidad
Añadir tests	test(almacenamiento): agregar tests para guardar y leer	Nueva característica de calidad
CI/CD	ci(github): configurar pipeline con GitHub Actions	Configuración de integración
Almacenamiento TXT	feat(almacenamiento): implementar guardado en TXT	Nueva funcionalidad de persistencia
Almacenamiento JSON	feat(almacenamiento): implementar guardado en JSON	Nueva funcionalidad de persistencia
Documentación README	docs: agregar README con instalación y uso	Documentación del proyecto
Corrección de bug	fix(niveles): corregir parsing de JSON en respuesta	Corrección de error

BLOQUE 7 — Resultado Final de la Empresa

El resultado final del proyecto debe incluir la interfaz gráfica (GUI) desarrollada con Tkinter, que muestra los análisis en tiempo real y confirma el guardado automático de resultados.



ANÁLISIS DE SENTIMIENTO - LOCAL

Texto a analizar

Me encanta este producto. La calidad es excelente y el servicio al cliente es maravilloso. Definitivamente lo recomendaría a mis amigos.

ANALIZAR SENTIMIENTO LIMPIAR GUARDAR

Resultados por Nivel | Análisis Detallado | Justificación & Recomendación | Historial

Nivel	Sentimiento	Polaridad	Intensidad
☒ Básico	POSITIVO	90.99%	-
☐ Intermedio	POSITIVO	0.9	ALTA
☐ Avanzado	POSITIVO	0.9	-

¿Qué significa la polaridad?

- ☒ POSITIVA (+0.00 a +1.00): El texto expresa emociones positivas
- ☐ NEGATIVA (-1.00 a -0.00): El texto expresa emociones negativas
- ☐ NEUTRAL (0.00): El texto no muestra emociones fuertes

☒ Análisis completado y guardado automáticamente

La imagen esta en el repo en la raiz. GitHub\NLP_Sentimientos\InterfazLlegada.png

La interfaz debe mostrar:

Componente de la interfaz	Descripción	Obligatorio
Área de texto de entrada	Campo para introducir el texto a analizar	Sí
Botón Analizar Sentimiento	Lanza el análisis en los 3 niveles	Sí
Botón Limpiar	Limpia todos los campos	Sí
Botón Guardar	Guarda manualmente en TXT y JSON	Sí
Pestaña Resultados por Nivel	Tabla con Básico / Intermedio / Avanzado	Sí
Pestaña Análisis Detallado	Emociones y polaridad extendida	Sí
Pestaña Justificación	Explicación del análisis + recomendación	Sí
Pestaña Historial	Lista de análisis guardados	Sí
Panel ¿Qué significa la polaridad?	Leyenda explicativa	Sí
Barra de estado	Confirma guardado automático	Sí

Interfaz Gráfica (GUI) con Tkinter

La aplicación incluye una interfaz gráfica desarrollada con **Tkinter** que:

Características implementadas:

Análisis en tiempo real

1. Campo de texto para entrada
2. Botones: Analizar, Limpiar, Guardar
3. Cuatro pestañas: Resultados, JSON, Justificación, Historial

Visualización de resultados

1. Tabla con niveles: Básico, Intermedio, Avanzado
2. Sentimiento: positivo/negativo/neutral
3. Polaridad: -1.00 a +1.00
4. Intensidad: alta/media/baja
5. Justificación del análisis avanzado



Análisis de Sentimiento



Historial automático

1. Carga análisis guardados previamente
2. Muestra timestamp y sentimiento
3. Persistencia automática (opcional con checkbox)

Guía de polaridad

1. □ Positiva (+0.00 a +1.00): emociones positivas
2. □ Negativa (-1.00 a -0.00): emociones negativas
3. ○ Neutral (0.00): sin emociones fuertes

Archivos de la GUI:

gui.py - Punto de entrada
src/gui/widgets.py - Componentes de interfaz
src/gui/tema.py - Configuración visual (estilo Windows)
src/menu.py - Menú de entrada (CLI + GUI)

Ejecución:

Desde menú
python main.py
Opción 1: Ejecutar GUI

Directamente
python gui.py

Imagen de la interfaz:

La captura de pantalla de la interfaz se encuentra en:

- InterfazLlegada.png (raíz del proyecto)
- Referenciada en README.md

☑ El guardado automático debe activarse tras cada análisis exitoso. La barra de estado debe mostrar: '✓
Análisis completado y guardado automáticamente' con la ruta del archivo generado.

BLOQUE 8 — Reflexión Integrada

▣ *Reflexión final: Describe el flujo profesional completo que acabas de implementar. ¿Qué aporta cada paso (análisis → modularización → tests → CI/CD → almacenamiento → docs)? ¿Qué hubiera pasado si te saltas alguno?*

8.1 — Flujo profesional completo

Describe con tus palabras qué aprendiste en cada fase y por qué el orden importa:

Lo que aprendí lo dejo en esta tabla bien explicada.

Fase	Qué aprendí	Por qué el orden importa
Análisis del código heredado	Identificar problemas de diseño: código monolítico, sin tests, sin persistencia	Es el punto de partida para planificar la refactorización
Modularización	SRP (Single Responsibility Principle), separación de responsabilidades, estructuras de paquetes	Permite escalar el proyecto sin que todo dependa de todo
Tests	pytest, casos felices/bordes/error, fixtures, mocking	Sin tests no hay garantía de calidad, los tests deben ir con el código
CI/CD	GitHub Actions, automatización, ruff, bandit, coverage	Cada commit se valida automáticamente, detección temprana de errores
Almacenamiento	Persistencia en TXT y JSON, timestamps, estructura de carpetas	Los resultados deben guardarse para auditoría e integración con otros sistemas
Documentación	README.md, docs/, arquitectura, commits semánticos	Sin documentación el proyecto muere, otros desarrolladores no pueden contribuir

Qué hubiera pasado si me saltara alguno?

- **Sin modularización:** Código imposible de mantener, miedo a hacer cambios
- **Sin tests:** Bugs en producción, regressions sin detectar
- **Sin CI/CD:** Errores humanos, código defectuoso llega a producción
- **Sin almacenamiento:** No hay auditoría, no se puede revisar historial
- **Sin documentación:** Proyecto inútil para otros, conocimiento perdido

8.2 — ¿Cómo usaste la IA como copiloto?

Para cada bloque, indica en qué momento pediste ayuda a la IA y cómo validaste su respuesta antes de aplicarla:

Bloque	¿Para qué usé la IA?	¿Cómo verifiqué su respuesta?
Bloque 2 — Modularización	Generar estructura de módulos,prompt profesional	Revisé que cumpliera SRP, probé imports
Bloque 3 — Tests	Generar templates de tests, casos de prueba	Añadí tests específicos para mi código, verifiqué coverage
Bloque 4 — CI/CD	Configuración de GitHub Actions, comandos	Probé el workflow en mi fork
Bloque 5 — Almacenamiento	Estructura de guardar.py y leer.py	Ejecuté los tests de persistencia
Bloque 6 — Documentación	Estructura de README, ejemplos	Verifiqué que la información fuera correcta

Validación de respuestas de IA:

- Revisé que el código generado cumpliera los requisitos
- Ejecuté los tests para verificar funcionalidad
- Ajusté el código cuando no cumplía los estándares del proyecto
- Documentación adaptada al contexto específico del proyecto

8.3 — Autoevaluación

Marca con una X tu nivel de confianza al finalizar el ejercicio:



Análisis de Sentimiento



Nivel	Descripción	¿Qué necesitarías repasar?
☐ 1	Necesito más práctica	
☐ 2	Lo entiendo con ayuda	
☐ 3	Lo aplico con seguridad	
☒ 4	Podría enseñarlo	Revision de mejores practicas de CI/CD

Mi nivel de confianza: 4 - El proyecto completo está implementado y funcionando.

Habilidades demostradas:

- ✓Análisis de código heredado
- ✓Refactorización modular
- ✓Tests unitarios con pytest (46 tests, 100% coverage)
- ✓CI/CD con GitHub Actions
- ✓Persistencia TXT/JSON
- ✓Documentación profesional
- ✓GUI con Tkinter
- ✓Uso de IA como copiloto

BLOQUE 9 — Reflexión MemoriaFinal.md

En una carpeta de MemoriaFinal, señalar en un archivo llamado [memoriaFinal.md](#) donde explicareis el porqué el código original está mal configurado y que cambios realizasteis

La memoria final la eh dejado dentro del repositorio

GitHub\NLP_Sentimientos\MemoriaFinal